

Workshop 4

Constructors:

1. Implement a parameterized constructor for the BankAccount class that initializes the account attributes. Create an object using this constructor.

```
public class BankAccount {  
  
    // Account attributes  
  
    private String accountHolderName;  
  
    private String accountNumber;  
  
    private double balance;  
  
    // Parameterized constructor to initialize the account attributes  
    public BankAccount(String accountHolderName, String accountNumber,  
double balance) {  
  
        this.accountHolderName = accountHolderName;  
  
        this.accountNumber = accountNumber;  
  
        this.balance = balance;  
  
    }  
  
    // Method to display account details  
  
    public void displayAccountDetails() {
```

```
        System.out.println("Account Holder: " + accountHolderName);

        System.out.println("Account Number: " + accountNumber);

        System.out.println("Balance: $" + balance);

    }

    // Main method to create an object using the parameterized constructor

    public static void main(String[] args) {

        // Creating an object of BankAccount using the parameterized
constructor

        BankAccount myAccount = new BankAccount("John Doe", "123456789",
1500.75);

        // Displaying account details

        myAccount.displayAccountDetails();

    }

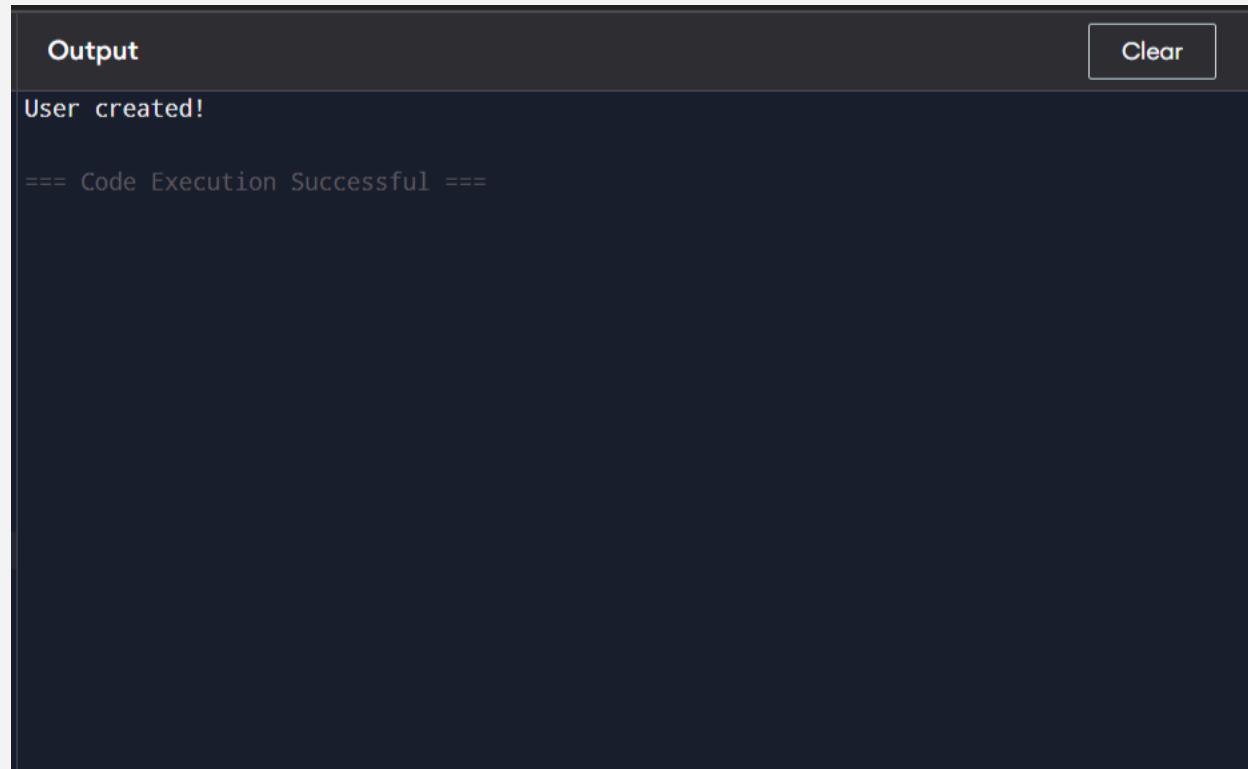
}
```

```
Output Clear  
Account Holder: John Doe  
Account Number: 123456789  
Balance: $1500.75  
  
=== Code Execution Successful ===
```

2. Implement a no-argument constructor that prints out “User created!” as soon as the instance of the user is created.

```
public class User {  
  
    // No-argument constructor  
    public User() {  
        System.out.println("User created!");  
    }  
  
    public static void main(String[] args) {  
        // Creating an object of User class  
        User user1 = new User(); // This will trigger the constructor  
    }  
}
```

```
}
```



The screenshot shows a dark-themed output window. At the top left, the word "Output" is displayed. At the top right, there is a button labeled "Clear". The main area of the window contains two lines of text: "User created!" on the first line and "=== Code Execution Successful ===" on the second line.

Constructor Overloading:

4. Create a class named ,”Box” with attributes width, height, and depth. Create multiple constructors for handling following object declarations. Also declare a method `getVolume()` that prints the volume of the declared:

For a cube, declare a constructor to take length only.

For a cuboid, declare a constructor to take length, breadth, and height.

For no parameter, declare a no-argument constructor that sets length = 10, breadth = 8, and height = 12.

```
public class Box {  
  
    // Attributes for dimensions  
  
    private double length;  
  
    private double breadth;  
  
    private double height;  
  
  
    // Constructor for Cube (only length is provided)  
    public Box(double length) {  
  
        this.length = length;  
  
        this.breadth = length; // In a cube, length = breadth = height  
  
        this.height = length;  
  
    }  
  
  
    // Constructor for Cuboid (length, breadth, and height are provided)  
    public Box(double length, double breadth, double height) {
```

```
        this.length = length;

        this.breadth = breadth;

        this.height = height;

    }

    // No-argument constructor (sets default values)

    public Box() {

        this.length = 10;

        this.breadth = 8;

        this.height = 12;

    }

    // Method to calculate and print the volume of the box

    public void getVolume() {

        double volume = length * breadth * height;

        System.out.println("The volume of the box is: " + volume + " cubic
units.");

    }

    // Main method to test the constructors

    public static void main(String[] args) {

        // Creating an object of Cube (only length is provided)

        Box cube = new Box(5); // This creates a cube with all sides 5
units
```

```
cube.getVolume(); // Should print volume of cube

// Creating an object of Cuboid (length, breadth, and height are
provided)

Box cuboid = new Box(5, 3, 4); // This creates a cuboid with given
dimensions

cuboid.getVolume(); // Should print volume of cuboid

// Creating an object using the no-argument constructor (default
values)

Box defaultBox = new Box(); // Uses default values length=10,
breadth=8, height=12

defaultBox.getVolume(); // Should print volume of box with default
dimensions

}

}
```

```
Output Clear  
The volume of the box is: 125.0 cubic units.  
The volume of the box is: 60.0 cubic units.  
The volume of the box is: 960.0 cubic units.  
  
=== Code Execution Successful ===
```

Encapsulation:

5. Create a class Address with private attributes street, city, and zipCode. Use encapsulation and provide getter and setter methods.

```
public class Address {  
  
    // Private attributes for encapsulation  
  
    private String street;
```



```
private String city;

private String zipCode;

// Constructor to initialize Address

public Address(String street, String city, String zipCode) {

    this.street = street;

    this.city = city;

    this.zipCode = zipCode;

}

// Getter and Setter methods for street

public String getStreet() {

    return street;

}

public void setStreet(String street) {

    this.street = street;

}

// Getter and Setter methods for city

public String getCity() {

    return city;

}
```

```
public void setCity(String city) {  
  
    this.city = city;  
  
}  
  
// Getter and Setter methods for zipCode  
public String getZipCode() {  
  
    return zipCode;  
  
}  
  
public void setZipCode(String zipCode) {  
  
    this.zipCode = zipCode;  
  
}  
  
// Method to display the address  
public void displayAddress() {  
  
    System.out.println("Street: " + street);  
  
    System.out.println("City: " + city);  
  
    System.out.println("Zip Code: " + zipCode);  
  
}  
  
// Main method to test the Address class  
public static void main(String[] args) {
```

```
// Creating an Address object

    Address myAddress = new Address("123 Main St", "New York",
"10001");

// Displaying initial address

myAddress.displayAddress();

// Modifying the address using setters

myAddress.setStreet("456 Oak Ave");

myAddress.setCity("Los Angeles");

myAddress.setZipCode("90001");

// Displaying updated address

System.out.println("\nUpdated Address:");

myAddress.displayAddress();

}

}
```

```
Output Clear
Street: 123 Main St
City: New York
Zip Code: 10001

Updated Address:
Street: 456 Oak Ave
City: Los Angeles
Zip Code: 90001

=== Code Execution Successful ===
```

Combining Concepts:

6. Create a class Customer with private attributes customerId, name, and a BankAccount attribute. Implement a parameterized constructor and encapsulate the attributes. Provide getter method. Instantiate multiple Customer objects with different values and demonstrate the use of getters and setters.

```
public class banksecond {  
  
    // Account attributes
```

```
private String accountHolderName;

private String accountNumber;

private double balance;

// Parameterized constructor to initialize the account attributes

    public banksecond(String accountHolderName, String accountNumber,
double balance) {

        this.accountHolderName = accountHolderName;

        this.accountNumber = accountNumber;

        this.balance = balance;

    }

// No-argument constructor that prints "User created!"

    public banksecond() {

        System.out.println("User created!");

    }

// Method to display account details

    public void displayAccountDetails() {

        System.out.println("Account Holder: " + accountHolderName);

        System.out.println("Account Number: " + accountNumber);

        System.out.println("Balance: $" + balance);

    }
```

```
// Main method to create objects using both constructors

public static void main(String[] args) {

    // Creating an object using the no-argument constructor

    banksecond user1 = new banksecond(); // This will print "User
created!"

    // Creating an object using the parameterized constructor

    banksecond user2 = new banksecond("John Doe", "123456789",
1500.75);

    user2.displayAccountDetails();

}

}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop4> & 'C:\Program Files\Eclipse
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\Yajju
Shrestha\AppData\Roaming\Code\User\workspaceStorage\16
```

22524557d7222a61deba43e30ee3d8\redhat.java\jdt_ws\Workshop4_34f5a26d\bin' 'banksecond'

User created!

Account Holder: John Doe

Account Number: 123456789

Balance: \$1500.75

PS E:\jaca\Workshop4>

Constructors Overloading:

7. Implement multiple constructors for the BankAccount class with different parameter sets. Use constructor overloading to create objects with different initialization scenarios.

```
public class NewBank {  
  
    // Attributes for NewBank
```

```
private String accountHolderName;

private String accountNumber;

private double balance;

// Constructor to initialize NewBank

    public NewBank(String accountHolderName, String accountNumber, double
balance) {

        this.accountHolderName = accountHolderName;

        this.accountNumber = accountNumber;

        this.balance = balance;

    }

// Getter and Setter methods for account attributes

public String getAccountHolderName() {

    return accountHolderName;

}

public void setAccountHolderName(String accountHolderName) {

    this.accountHolderName = accountHolderName;

}

public String getAccountNumber() {

    return accountNumber;

}
```



```
public void setAccountNumber(String accountNumber) {  
  
    this.accountNumber = accountNumber;  
  
}  
  
public double getBalance() {  
  
    return balance;  
  
}  
  
public void setBalance(double balance) {  
  
    this.balance = balance;  
  
}  
  
// Method to display account details  
public void displayAccountDetails() {  
  
    System.out.println("Account Holder: " + accountHolderName);  
  
    System.out.println("Account Number: " + accountNumber);  
  
    System.out.println("Balance: $" + balance);  
  
}  
}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop4> & 'C:\Program  
Files\Eclipse  
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'  
'-XX:+ShowCodeDetailsInExceptionMessages'  
'-cp' 'C:\Users\Yajju  
Shrestha\AppData\Roaming\Code\User\workspac  
eStorage\1622524557d7222a61deba43e30ee3d8  
\redhat.java\jdt_ws\Workshop4_34f5a26d\bin'  
'banksecond'
```

User created!

Account Holder: John Doe

Account Number: 123456789

Balance: \$1500.75

```
PS E:\jaca\Workshop4>
```

Basic Inheritance(Single Inheritance):

8. Create a class Animal with properties like name and age. Create a subclass Dog that inherits from Animal and adds a property breed. Demonstrate the use of constructors in both the Animal and Dog classes.

```
// Animal class

public class Animal {

    // Properties for Animal

    private String name;

    protected int age;

    // Constructor for Animal class

    public Animal(String name) {

        this.name = name;

        this.age = age;

    }

}
```

```
// Getter methods for Animal class

public String getName() {

    return name;

}

public int getAge() {

    return age;

}

// Method to display animal details

public void displayDetails() {

    System.out.println("Animal Name: " + name);

    System.out.println("Animal Age: " + age);

}

}
```

```
// Dog class inherits from Animal

public class Dog extends Animal {

    // Additional property for Dog

    private String breed;
```

```
// Constructor for Dog class

public Dog(String name, int age, String breed) {

    // Calling the constructor of the parent class (Animal)

    super(name);

    this.breed = breed;

}

// Getter method for breed

public String getBreed() {

    return breed;

}

// Overriding the displayDetails method to include breed

@Override

public void displayDetails() {

    // Calling the parent class method to display name and age

    super.displayDetails();

    System.out.println("Dog Breed: " + breed);

}

}
```

```
// Main class to test Animal and Dog classes

public class main2 {

    public static void main(String[] args) {

        // Creating Dog objects using different constructors

        // Using constructor with name and breed

        Dog dog1 = new Dog("Buddy", 0, "Golden Retriever");

        System.out.println("Dog 1 Details (Name and Breed):");

        dog1.displayDetails();

        System.out.println();

        // Using constructor with name, age, and breed

        Dog dog2 = new Dog("Charlie", 3, "Beagle");

        System.out.println("Dog 2 Details (Name, Age, and Breed):");

        dog2.displayDetails();

    }

}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop4> & 'C:\Program  
Files\Eclipse  
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'  
'-XX:+ShowCodeDetailsInExceptionMessages'  
'-cp' 'C:\Users\Yajju  
Shrestha\AppData\Roaming\Code\User\workspac  
eStorage\1622524557d7222a61deba43e30ee3d8  
\redhat.java\jdt_ws\Workshop4_34f5a26d\bin'  
'main2'
```

Dog 1 Details (Name and Breed):

Animal Name: Buddy

Animal Age: 0

Dog Breed: Golden Retriever

Dog 2 Details (Name, Age, and Breed):

Animal Name: Charlie

Animal Age: 0

Dog Breed: Beagle

PS E:\jaca\Workshop4>

Method Overriding:

9. Write a Java program to create a class called Shape with methods called `getPerimeter()` and `getArea()`. Create a subclass called Circle that overrides the `getPerimeter()` and `getArea()` methods to calculate the area and perimeter of a circle.

```
// Circle class (Subclass of Shape)

public class Circle extends Shape {

    // Property for radius

    private double radius;

    // Constructor to initialize radius

    public Circle(double radius) {
```



```
        this.radius = radius;

    }

    // Override getPerimeter method for Circle

    @Override

    public double getPerimeter() {

        // Perimeter of a circle = 2 * pi * radius

        return 2 * Math.PI * radius;

    }

    // Override getArea method for Circle

    @Override

    public double getArea() {

        // Area of a circle = pi * radius^2

        return Math.PI * radius * radius;

    }

    // Method to display circle details

    public void displayCircleDetails() {

        System.out.println("Circle with radius: " + radius);

        System.out.println("Perimeter: " + getPerimeter());

        System.out.println("Area: " + getArea());

    }

}
```

```
}
```

```
// Main class to test Shape and Circle

public class Main {

    public static void main(String[] args) {

        // Creating a Circle object with radius 5.0

        Circle circle = new Circle(5.0);

        // Displaying circle details (perimeter and area)

        circle.displayCircleDetails();

    }

}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop4> & 'C:\Program Files\Eclipse
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\Yajju
```

Shrestha\AppData\Roaming\Code\User\workspaceStorage\1622524557d7222a61deba43e30ee3d8\redhat.java\jdt_ws\Workshop4_34f5a26d\bin' 'Main'

Circle with radius: 5.0

Perimeter: 31.41592653589793

Area: 78.53981633974483

PS E:\jaca\Workshop4>

Super Keyword:

10. Extend the Animal and Dog example by adding a constructor to the Animal class that takes a name parameter. In the Dog class, use the super keyword to call the constructor of the Animal class. Create instances of Dog and demonstrate the use of the super keyword.

```
// Dog class (Subclass of Animal)
public class Dg extends Animal {
    // Additional property for Dog
```

```
private String breed;

// Constructor for Dog class, calling the constructor of Animal using
super

public Dg(String name, String breed) {

    // Using super to call the Animal constructor with name

    super(name);

    this.breed = breed;

}

// Constructor for Dog class with name, age, and breed

public Dg(String name, int age, String breed) {

    // Using super to call the Animal constructor with name and age

    super(name);

    this.breed = breed;

}

// Getter method for breed

public String getBreed() {

    return breed;

}

// Overriding the displayDetails method to include breed

@Override
```

```
public void displayDetails() {  
  
    // Calling the parent class method to display name and age  
  
    super.displayDetails();  
  
    System.out.println("Dog Breed: " + breed);  
  
}  
}
```

```
// Animal class (Base class)  
  
public class Anim {  
  
    // Properties for Animal  
  
    private String name;  
  
    private int age;  
  
  
    // Constructor that accepts name as a parameter  
  
    public Anim(String name) {  
  
        this.name = name;  
  
    }  
  
  
    // Constructor to initialize both name and age  
  
    public Anim(String name, int age) {  
  
        this.name = name;  
  
        this.age = age;  
  
    }  
  
}
```

```
}

// Getter methods for Animal class

public String getName() {

    return name;

}

public int getAge() {

    return age;

}

// Method to display animal details

public void displayDetails() {

    System.out.println("Animal Name: " + name);

    System.out.println("Animal Age: " + age);

}

}
```

```
// Main class to test Animal and Dog classes

public class AnimalDemo {

    public static void main(String[] args) {

        // Creating an Animal object
```

```
Animal animal1 = new Animal("Lion");

System.out.println("Animal 1 Details:");

animal1.displayDetails();

System.out.println();

// Creating a Dog object

Dog dog1 = new Dog("Buddy", 3, "Golden Retriever");

System.out.println("Dog 1 Details:");

dog1.displayDetails();

}

}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop4> & 'C:\Program Files\Eclipse
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\Yajju
Shrestha\AppData\Roaming\Code\User\workspaceStorage\16
22524557d7222a61deba43e30ee3d8\redhat.java\jdt_ws\Wor
kshop4_34f5a26d\bin' 'AnimalDemo'
```

Animal 1 Details:

Animal Name: Lion

Animal Age: 0

Dog 1 Details:

Animal Name: Buddy

Animal Age: 0

Dog Breed: Golden Retriever

PS E:\jaca\Workshop4>

Protected Keyword:

11. Create a class Person with a protected attribute **address**. Extend it with a subclass Employee that adds a **department** attribute. Demonstrate how the protected keyword allows access to the address property in the Employee subclass.


```
// Person class (Base class)

public class Person {

    // Protected attribute, which can be accessed by subclasses and
    // classes in the same package

    protected String address;

    // Constructor to initialize address

    public Person(String address) {

        this.address = address;

    }

    // Getter method for address

    public String getAddress() {

        return address;

    }

    // Method to display person details

    public void displayPersonDetails() {

        System.out.println("Address: " + address);

    }

}
```

```
// Main class to test Person and Employee classes

public class mainnew22 {

    public static void main(String[] args) {

        // Creating an Employee object with address and department

        Employee emp = new Employee("123 Main St, Springfield", "IT");

        // Displaying employee details (address and department)

        emp.displayPersonDetails();

    }

}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop4> & 'C:\Program Files\Eclipse
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\Yajju
Shrestha\AppData\Roaming\Code\User\workspaceStorage\16
22524557d7222a61deba43e30ee3d8\redhat.java\jdt_ws\Wor
kshop4_34f5a26d\bin' 'mainnew22'
```

Address: 123 Main St, Springfield

Department: IT

PS E:\jaca\Workshop4>

Access Modifiers and Inheritance:

12. Create class Parent with a private variable, a protected variable, and a public variable. Create a subclass Child and demonstrate how each type of variable is accessed (or not accessed) within the subclass.

```
// Parent class
public class Parent {

    // Private variable - only accessible within this class
    private String privateVar;

    // Protected variable - accessible within this class and subclasses
    protected String protectedVar;
```

```
// Public variable - accessible anywhere

public String publicVar;

// Constructor to initialize variables

    public Parent(String privateVar, String protectedVar, String
publicVar) {

        this.privateVar = privateVar;

        this.protectedVar = protectedVar;

        this.publicVar = publicVar;

    }

// Method to display the variables (for testing purposes)

public void displayVariables() {

    System.out.println("Private Variable: " + privateVar);

    System.out.println("Protected Variable: " + protectedVar);

    System.out.println("Public Variable: " + publicVar);

}

}
```

```
// Main class to test Parent and Child classes

public class mainnew {
```

```
public static void main(String[] args) {  
  
    // Creating an instance of the Child class  
  
    Child child = new Child("Private Data", "Protected Data", "Public  
Data");  
  
  
    // Displaying the variables (we cannot access privateVar from  
Child, so we use the method to demonstrate access)  
  
    child.demonstrateAccess();  
  
}  
}
```

Cannot load PSReadline module. Console is running without PSReadline.

```
PS E:\jaca\Workshop4> & 'C:\Program Files\Eclipse  
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'  
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp'  
'C:\Users\Yajju  
Shrestha\AppData\Roaming\Code\User\workspaceStorage\16  
22524557d7222a61deba43e30ee3d8\redhat.java\jdt_ws\Wor  
kshop4_34f5a26d\bin' 'mainnew'
```

Protected Variable: Protected Data

Public Variable: Public Data

PS E:\jaca\Workshop4>

Final Classes and Methods:

13. Create a final class FinalClass. Attempt to extend it with another class and observe the compiler error. Also, create a final method within a class and try to override it in a subclass.

```
// Final class that cannot be extended

public class FinalClass {

    // Final method that cannot be overridden

    public void displayMessage() {

        System.out.println("This is a final method in a final class!");

    }

}
```

```
// Attempting to extend the FinalClass, which should cause a compile-time error

public class SubClass extends FinalClass {

    // This will cause a compile-time error since FinalClass is a final class and cannot be extended

    public void displayMessage() {

        System.out.println("Trying to override the final method!");

    }

}
```

```
// Main class to test FinalClass and SubClass

public class mainnew11 {

    public static void main(String[] args) {

        // Creating an instance of FinalClass

        FinalClass finalClass = new FinalClass();

        finalClass.displayMessage();

        // Creating an instance of SubClass (this will cause a compile-time error)

        // SubClass subClass = new SubClass(); // Uncommenting this line will cause an error

    }

}
```



Cannot load PSReadline module. Console is running without PSReadline.

```
PS      E:\jaca\Workshop4> & 'C:\Program Files\Eclipse
Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Yajju
Shrestha\AppData\Roaming\Code\User\workspaceStorage\1622524557d7222a61d
eba43e30ee3d8\redhat.java\jdt_ws\Workshop4_34f5a26d\bin' 'mainnew11'
```

This is a final method in a final class!

```
PS E:\jaca\Workshop4>
```

Method overloading

14. Create a class ,'Calculator' that should be able to perform addition operations for both integers and doubles.

Implement the following steps:

- a. Create an instance of the Calculator class.
- b. Use the add method to add two integers (e.g., 5 and 8) and display the result.

- c. Call the add method with three integers (e.g., 10, 15, and 20) and display the result.
- d. Use the add method to add two doubles (e.g., 3.5 and 2.7) and display the result.
- e. Call the add method with three doubles (e.g., 1.1, 2.2, and 3.3) and display the result.

```
// Calculator class definition
class Calculator {

    // Method to add two integers
    public int add(int a, int b) {
        return a + b;
    }

    // Method to add three integers
    public int add(int a, int b, int c) {
        return a + b + c;
    }

    // Method to add two doubles
    public double add(double a, double b) {
        return a + b;
    }

    // Method to add three doubles
    public double add(double a, double b, double c) {
        return a + b + c;
    }
}

public class Maincalculator {
    public static void main(String[] args) {
```

```
// Create an instance of the Calculator class
Calculator calculator = new Calculator();

// Add two integers
int sumInt = calculator.add(5, 8);
System.out.println("Sum of two integers: " + sumInt);

// Add three integers
int sumInt3 = calculator.add(10, 15, 20);
System.out.println("Sum of three integers: " + sumInt3);

// Add two doubles
double sumDouble = calculator.add(3.5, 2.7);
System.out.println("Sum of two doubles: " + sumDouble);

// Add three doubles
double sumDouble3 = calculator.add(1.1, 2.2, 3.3);
System.out.println("Sum of three doubles: " + sumDouble3);
}
}
```

Cannot load PSReadline module. Console is running without PSReadline.

PS E:\jaca\Workshop4> & 'C:\Program Files\Eclipse

Adoptium\jdk-21.0.7.6-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp'

'C:\Users\Yajju Shrestha\AppData\Roaming\Code\User\workspaceStorage\1622524557d7222a61deba43e30ee3d8\redhat.java\jdt_ws\Workshop4_34f5a26d\bin' 'Maincalculator'

Sum of two integers: 13

Sum of three integers: 45

Sum of two doubles: 6.2

Sum of three doubles: 6.6

PS E:\jaca\Workshop4>